

手摸手实现用户基础组件库

组件地址

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-user-spring-boot-starter</artifactId>
  <version>${project.version}</version>
</dependency>
```

组件概述

1. 通过 JWT 生成用户登录唯一凭证。
2. 封装当前请求用户上下文。
3. 拦截带 Token 的请求，并将 Token 解析放入用户上下文中。

组件功能

JWT Token 生成器

通过 JWT 生成用户唯一 Token 凭证，并提供反解析 Token 凭证位用户信息方法。应用于用户服务以及网关服务。

```
package org.opengoofy.index12306.frameworks.starter.user.toolkit;

import com.alibaba.fastjson2.JSON;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.ExpiredJwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import lombok.extern.slf4j.Slf4j;
import org.opengoofy.index12306.frameworks.starter.user.core.UserInfoDTO;
import org.springframework.util.StringUtils;
```

```

import java.util.Date;
import java.util.HashMap;
import java.util.Map;

import static
org.opengoofy.index12306.framework.starter.bases.constant.UserConstant.REAL_NAME_KEY;
import static
org.opengoofy.index12306.framework.starter.bases.constant.UserConstant.USER_ID_KEY;
import static
org.opengoofy.index12306.framework.starter.bases.constant.UserConstant.USER_NAME_KEY;

/**
 * JWT 工具类
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j
public final class JWTUtil {

    private static final long EXPIRATION = 86400L;
    public static final String TOKEN_PREFIX = "Bearer ";
    public static final String ISS = "index12306";
    public static final String SECRET =
"SecretKey039245678901232039487623456783092349288901402967890140939827";

    /**
     * 生成用户 Token
     *
     * @param userInfo 用户信息
     * @return 用户访问 Token
     */
    public static String generateAccessToken(UserInfoDTO userInfo) {
        Map<String, Object> customerUserMap = new HashMap<>();
        customerUserMap.put(USER_ID_KEY, userInfo.getUserId());
        customerUserMap.put(USER_NAME_KEY, userInfo.getUsername());
        customerUserMap.put(REAL_NAME_KEY, userInfo.getRealName());
        String jwtToken = Jwts.builder()
            .signWith(SignatureAlgorithm.HS512, SECRET)
            .setIssuedAt(new Date())
            .setIssuer(ISS)
            .setSubject(JSON.toJSONString(customerUserMap))
    }

```

```

        .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION * 1000))
        .compact();
    return TOKEN_PREFIX + jwtToken;
}

/**
 * 解析用户 Token
 *
 * @param jwtToken 用户访问 Token
 * @return 用户信息
 */
public static UserInfoDTO parseJwtToken(String jwtToken) {
    if (StringUtils.hasText(jwtToken)) {
        String actualJwtToken = jwtToken.replace(TOKEN_PREFIX, "");
        try {
            Claims claims =
Jwts.parser().setSigningKey(SECRET).parseClaimsJws(actualJwtToken).getBody();
            Date expiration = claims.getExpiration();
            if (expiration.after(new Date())) {
                String subject = claims.getSubject();
                return JSON.parseObject(subject, UserInfoDTO.class);
            }
        } catch (ExpiredJwtException ignored) {
        } catch (Exception ex) {
            log.error("JWT Token解析失败，请检查", ex);
        }
    }
    return null;
}
}

```

封装用户上下文

1) 定义用户参数实体。

仅定义有且必须的，当前想到了三个字段，后续可能还会补充。一切以代码实际行动为准。

```

package org.opengoofy.index12306.frameworks.starter.user.core;

import lombok.AllArgsConstructor;

```

```

import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

/**
 * 用户信息实体
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserInfoDTO {

    /**
     * 用户 ID
     */
    private String userId;

    /**
     * 用户名
     */
    private String username;

    /**
     * 真实姓名
     */
    private String realName;
}

```

2) 定义用户参数上下文。

```

package org.opengoofy.index12306.frameworks.starter.user.core;

import com.alibaba.ttl.TransmittableThreadLocal;

import java.util.Optional;

/**
 * 用户上下文

```

```

*
* @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
*/
public final class UserContext {

    private static final ThreadLocal<UserInfoDTO> USER_THREAD_LOCAL = new
TransmittableThreadLocal<>();

    /**
     * 设置用户至上下文
     *
     * @param user 用户详情信息
     */
    public static void setUser(UserInfoDTO user) {
        USER_THREAD_LOCAL.set(user);
    }

    /**
     * 获取上下文中用户 ID
     *
     * @return 用户 ID
     */
    public static String getUserId() {
        UserInfoDTO userInfoDTO = USER_THREAD_LOCAL.get();
        return Optional.ofNullable(userInfoDTO).map(UserInfoDTO::getUserId).orElse(null);
    }

    /**
     * 获取上下文中用户名称
     *
     * @return 用户名称
     */
    public static String getUsername() {
        UserInfoDTO userInfoDTO = USER_THREAD_LOCAL.get();
        return
Optional.ofNullable(userInfoDTO).map(UserInfoDTO::getUsername).orElse(null);
    }

    /**
     * 获取上下文中用户真实姓名
     *

```

```

    * @return 用户真实姓名
    */
    public static String getRealName() {
        UserInfoDTO userInfoDTO = USER_THREAD_LOCAL.get();
        return
Optional.ofNullable(userInfoDTO).map(UserInfoDTO::getRealName).orElse(null);
    }

    /**
     * 清理用户上下文
     */
    public static void removeUser() {
        USER_THREAD_LOCAL.remove();
    }
}

```

使用阿里 TTL 安全的 ThreadLocal 作为存储容器，绑定当前请求用户参数。

[GitHub - alibaba/transmittable-thread-local](#):  TransmittableThreadLocal (TTL), the missing Java™ std lib(simple & 0-dependency) for framework/middleware, provide an enhanced InheritableThreadLocal that transmits values between threads even using thread pooling components.

用户上下文拦截器

添加用户上下文过滤器，如果 HTTP 请求 Header 中包含用户信息，则进行解析并放入 `UserContext`。

```

package org.opengoofy.index12306.frameworks.starter.user.core;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.opengoofy.index12306.framework.starter.bases.constant.UserConstant;
import org.springframework.util.StringUtils;

import java.io.IOException;

```

```
import java.net.URLDecoder;

import static java.nio.charset.StandardCharsets.UTF_8;

/**
 * 用户信息传输过滤器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public class UserTransmitFilter implements Filter {

    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse,
        FilterChain filterChain) throws IOException, ServletException {
        HttpServletRequest httpRequest = (HttpServletRequest) servletRequest;
        String userId = httpRequest.getHeader(UserConstant.USER_ID_KEY);
        if (StringUtils.hasText(userId)) {
            String userName = httpRequest.getHeader(UserConstant.USER_NAME_KEY);
            String realName = httpRequest.getHeader(UserConstant.REAL_NAME_KEY);
            if (StringUtils.hasText(userName)) {
                userName = URLDecoder.decode(userName, UTF_8);
            }
            if (StringUtils.hasText(realName)) {
                realName = URLDecoder.decode(realName, UTF_8);
            }
            UserInfoDTO userInfoDTO = UserInfoDTO.builder()
                .userId(userId)
                .username(userName)
                .realName(realName)
                .build();
            UserContext.setUser(userInfoDTO);
        }
        try {
            filterChain.doFilter(servletRequest, servletResponse);
        } finally {
            UserContext.removeUser();
        }
    }
}
```

更新: 2023-07-17 15:14:38

原文: <<https://www.yuque.com/magestack/12306/afxyadhwxuycxpkx>>